Reinforcement Learning (DSAI 402)

Lecture 6: Monte Carlo Control and TD Prediction

Prof. Mohamed Ghalwash
mghalwash@zewailcity.edu.eg

Zewail City University

Lecture 6, Monday 2 November 2026

Agenda

Objectives: you should be able to find a good policy from played episodes with MC control, explain why an ϵ-greedy policy is needed, run the TD(0) update by hand and decide whether DP, MC or TD fits a given problem.

- Finish MC: from prediction to control with q and ϵ-greedy policies
- Why wait for the end of the episode?
- TD(0) prediction: the TD target, the TD error and bootstrapping
- Apply it: the fifth application paper, TD-Gammon
- Practice it: turn six problems into RL problems

Part 1: Monte Carlo Control

Where We Stopped: MC Prediction

To estimate v_π, play many episodes with π and average the returns seen after each state.

$$V(S_t) \leftarrow V(S_t) + \frac{1}{N(S_t)}\big(G_t - V(S_t)\big)$$

- No model, only played episodes.
- In the bridge walk, 60 walks gave rough values and 10,000 walks gave values close to $\frac{1}{6}, \frac{2}{6}, \ldots, \frac{5}{6}$.

Prediction tells us **how good** a policy is. Today we first want a **better** policy.

What Is MC Control?

	Prediction	Control
Question	How good is a given policy π?	What is the best policy?
Output	v_π or q_π	A policy, as close to π_* as we can get
With the model (DP)	Policy evaluation	Policy iteration, value iteration
From episodes (MC)	MC prediction (last week)	**MC control (today)**

MC control finds a good policy from played episodes, without the model. It repeats two steps:

1. **Evaluate**: play episodes and average their returns to estimate the action values $q(s, a)$.

Why Estimate q and Not v?

In DP, we improved a policy by acting greedily with respect to v:

$$\pi'(s) = \arg\max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma\, v(s') \right]$$

This needs the model $p(s', r \mid s, a)$, which MC does not have.

Why Estimate q and Not v?

In DP, we improved a policy by acting greedily with respect to v:

$$\pi'(s) = \arg\max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma\, v(s') \right]$$

This needs the model $p(s', r \mid s, a)$, which MC does not have.

With q, the greedy step needs no model:

$$\pi'(s) = \arg\max_a q(s, a)$$

Important
Without a model, we learn action values, i.e. one number q(s, a) for each state and action pair.

MC Estimation of q_π

The same idea as last week, with a pair (s, a) in place of a state s.

$$q_\pi(s, a) = \mathbb{E}_\pi \left[G_t \mid S_t = s, A_t = a \right] \approx \text{ average of the returns after } (s, a)$$

- A visit is now a visit to the pair, i.e. the agent was in s **and** took a.
- First-visit and every-visit work as before.

MC Estimation of q_π

The same idea as last week, with a pair (s, a) in place of a state s.

$$q_\pi(s, a) = \mathbb{E}_\pi \left[G_t \mid S_t = s, A_t = a \right] \approx \text{ average of the returns after } (s, a)$$

- A visit is now a visit to the pair, i.e. the agent was in s **and** took a.
- First-visit and every-visit work as before.

The problem: if π is greedy, it always picks the same action in s. The other actions are never tried, so their q values are never estimated, and we can never find out that one of them is better.

Two Ways to Keep Trying Every Action

	Exploring starts	ϵ-greedy
Idea	Start each episode from a random pair (s, a)	Most of the time take the best action, sometimes take a random one
Needs	Control over where episodes start	Nothing extra
Real world	Hard. You cannot start a patient, a plane or a chip at any state you like	Works anywhere the agent can choose its actions

We use ϵ-greedy from now on.

The ϵ-Greedy Policy

With $|\mathcal{A}|$ actions and a small ϵ (e.g. 0.1):

- With probability $1 - \epsilon$, take the greedy action $\arg\max_a Q(s, a)$.
- With probability ϵ, take an action at random. This random action can also be the greedy one.

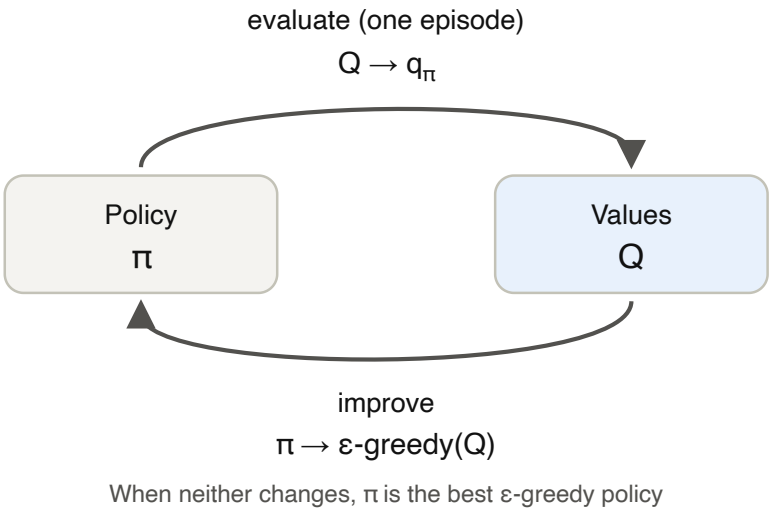
$$\pi(a \mid s) = \begin{cases} 1 - \epsilon + \dfrac{\epsilon}{|\mathcal{A}|} & \text{greedy } a \\ \dfrac{\epsilon}{|\mathcal{A}|} & \text{any other } a \end{cases}$$

Example with 2 actions and $\epsilon = 0.1$:

Action	$Q(s, a)$	$\pi(a \mid s)$
Left	0.3	$0.9 + 0.05 = 0.95$
Right	-0.1	0.05

Every action keeps a chance of at least $\epsilon/|\mathcal{A}|$, so in every state we visit, every action keeps being tried.

MC Control Is GPI



After **each episode**:

1. **Evaluate**: update $Q(s, a)$ for every pair in the episode with its return.
2. **Improve**: in every state of the episode, make the policy ϵ-greedy with respect to the new Q.

Policy iteration evaluates until the values settle, value iteration evaluates for one sweep and MC control evaluates for one episode.

On-Policy First-Visit MC Control

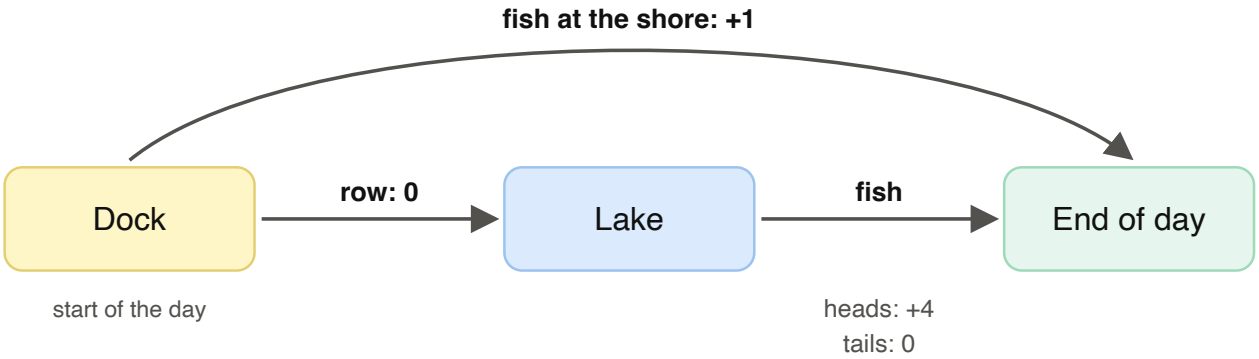
```
Input: small ε > 0
Set Q(s,a) = 0 and N(s,a) = 0 for every pair
π = ε-greedy with respect to Q

repeat for many episodes:
    play one episode with π:  S_0, A_0, R_1, ..., S_{T-1}, A_{T-1}, R_T
    G = 0
    for t = T-1, T-2, ..., 0:
        G = γ G + R_{t+1}
        if (S_t, A_t) does not appear earlier in the episode:      # first visit
            N(S_t,A_t) = N(S_t,A_t) + 1
            Q(S_t,A_t) = Q(S_t,A_t) + (G - Q(S_t,A_t)) / N(S_t,A_t)
            π(S_t) = ε-greedy with respect to Q(S_t, ·)

return π, Q
```

- The only changes from last week: $Q(s,a)$ in place of $V(s)$, and the policy changes after every episode.
- **On-policy**: the policy that plays the episodes is the same policy that we improve.

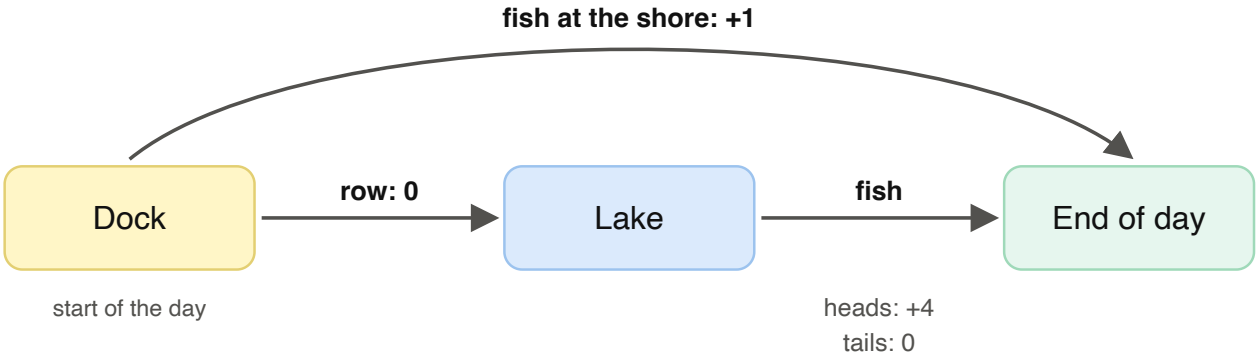
Example: The Fishing Trip



- Each day is one episode, with $\gamma = 1$. At the lake, a coin decides the catch.
- The agent does not know the coin. It only goes fishing and counts its fish.

Which choice is better at the dock?

Example: The Fishing Trip



- Each day is one episode, with $\gamma = 1$. At the lake, a coin decides the catch.
- The agent does not know the coin. It only goes fishing and counts its fish.

Which choice is better at the dock?

$q(\text{dock}, \text{shore}) = 1$ and $q(\text{dock}, \text{row}) = 0 + \frac{1}{2} \cdot 4 + \frac{1}{2} \cdot 0 = 2$. Rowing is better, but the agent has to find this out from its own days.

Pure Greedy Gets Stuck

MC control with no exploration. All Q values start at 0 and ties are broken at random. After each day, $Q(\text{dock}, a)$ is the average of the returns after choosing a.

Day	Choice at the dock	Coin	Return	$Q(\text{dock}, \text{shore})$	$Q(\text{dock}, \text{row})$

Pure Greedy Gets Stuck

MC control with no exploration. All Q values start at 0 and ties are broken at random. After each day, $Q(\text{dock}, a)$ is the average of the returns after choosing a.

Day	Choice at the dock	Coin	Return	$Q(\text{dock}, \text{shore})$	$Q(\text{dock}, \text{row})$
1	row (a tie, picked at random)	tails	0	0	0

Pure Greedy Gets Stuck

MC control with no exploration. All Q values start at 0 and ties are broken at random. After each day, $Q(\text{dock}, a)$ is the average of the returns after choosing a.

Day	Choice at the dock	Coin	Return	$Q(\text{dock}, \text{shore})$	$Q(\text{dock}, \text{row})$
1	row (a tie, picked at random)	tails	0	0	0
2	shore (a tie, picked at random)		1	1	0

Pure Greedy Gets Stuck

MC control with no exploration. All Q values start at 0 and ties are broken at random. After each day, $Q(\text{dock}, a)$ is the average of the returns after choosing a.

Day	Choice at the dock	Coin	Return	$Q(\text{dock}, \text{shore})$	$Q(\text{dock}, \text{row})$
1	row (a tie, picked at random)	tails	0	0	0
2	shore (a tie, picked at random)		1	1	0
3	shore (greedy, since 1 > 0)		1	1	0

Pure Greedy Gets Stuck

MC control with no exploration. All Q values start at 0 and ties are broken at random. After each day, $Q(\text{dock}, a)$ is the average of the returns after choosing a.

Day	Choice at the dock	Coin	Return	$Q(\text{dock}, \text{shore})$	$Q(\text{dock}, \text{row})$
1	row (a tie, picked at random)	tails	0	0	0
2	shore (a tie, picked at random)		1	1	0
3	shore (greedy, since 1 > 0)		1	1	0
4, 5, ...	shore (greedy)		1	1	0

Pure Greedy Gets Stuck

MC control with no exploration. All Q values start at 0 and ties are broken at random. After each day, $Q(\text{dock}, a)$ is the average of the returns after choosing a.

Day	Choice at the dock	Coin	Return	$Q(\text{dock}, \text{shore})$	$Q(\text{dock}, \text{row})$
1	row (a tie, picked at random)	tails	0	0	0
2	shore (a tie, picked at random)		1	1	0
3	shore (greedy, since 1 > 0)		1	1	0
4, 5, ...	shore (greedy)		1	1	0

One unlucky coin on day 1, and the agent never rows again. In 100,000 simulated runs, pure greedy settles on rowing in only 1 run out of 3.

ϵ-Greedy Keeps Trying

Now with $\epsilon = 0.1$. Days 1 to 3 go as before, so $Q(\text{dock}, \text{shore}) = 1$ and $Q(\text{dock}, \text{row}) = 0$. On day 4, the agent makes a random move.

Day	Choice at the dock	Coin	Return	$Q(\text{dock}, \text{shore})$	$Q(\text{dock}, \text{row})$

ϵ-Greedy Keeps Trying

Now with $\epsilon = 0.1$. Days 1 to 3 go as before, so $Q(\text{dock}, \text{shore}) = 1$ and $Q(\text{dock}, \text{row}) = 0$. On day 4, the agent makes a random move.

Day	Choice at the dock	Coin	Return	$Q(\text{dock}, \text{shore})$	$Q(\text{dock}, \text{row})$
4	row (random move)	heads	4	1	$(0 + 4)/2 = 2$

ϵ-Greedy Keeps Trying

Now with $\epsilon = 0.1$. Days 1 to 3 go as before, so $Q(\text{dock}, \text{shore}) = 1$ and $Q(\text{dock}, \text{row}) = 0$. On day 4, the agent makes a random move.

Day	Choice at the dock	Coin	Return	$Q(\text{dock}, \text{shore})$	$Q(\text{dock}, \text{row})$
4	row (random move)	heads	4	1	$(0 + 4)/2 = 2$
5	row (greedy, since 2 > 1)	tails	0	1	$(0 + 4 + 0)/3 \approx 1.33$

ϵ-Greedy Keeps Trying

Now with $\epsilon = 0.1$. Days 1 to 3 go as before, so $Q(\text{dock}, \text{shore}) = 1$ and $Q(\text{dock}, \text{row}) = 0$. On day 4, the agent makes a random move.

Day	Choice at the dock	Coin	Return	$Q(\text{dock}, \text{shore})$	$Q(\text{dock}, \text{row})$
4	row (random move)	heads	4	1	$(0 + 4)/2 = 2$
5	row (greedy, since 2 > 1)	tails	0	1	$(0 + 4 + 0)/3 \approx 1.33$
6	row (greedy, since 1.33 > 1)	heads	4	1	$(0 + 4 + 0 + 4)/4 = 2$

ϵ-Greedy Keeps Trying

Now with $\epsilon = 0.1$. Days 1 to 3 go as before, so $Q(\text{dock}, \text{shore}) = 1$ and $Q(\text{dock}, \text{row}) = 0$. On day 4, the agent makes a random move.

Day	Choice at the dock	Coin	Return	$Q(\text{dock}, \text{shore})$	$Q(\text{dock}, \text{row})$
4	row (random move)	heads	4	1	$(0 + 4)/2 = 2$
5	row (greedy, since 2 > 1)	tails	0	1	$(0 + 4 + 0)/3 \approx 1.33$
6	row (greedy, since 1.33 > 1)	heads	4	1	$(0 + 4 + 0 + 4)/4 = 2$

The random moves keep testing both choices, so both averages move toward the true values, 1 and 2. In 100,000 simulated runs, 88% of runs row after 100 days, and all but 7 row after 1,000 days.

What Does ϵ Cost?

- The agent keeps making random moves, so it loses a little. On the fishing trip with $\epsilon = 0.1$, it still fishes at the shore on 5% of the days. It catches $0.95 \times 2 + 0.05 \times 1 = 1.95$ fish per day on average, not 2.
- MC control finds the best policy **among ϵ-greedy policies**, not the best policy overall.
- The usual fix in practice: shrink ϵ slowly over time, or play the greedy policy once learning is done.

> **Tip**
> Next lecture, Q-learning learns about the greedy policy while it plays an exploring policy. That is called off-policy learning.

Learn More

Course Homepage